

Basic course

Bus devices

KNX Association

Table of Contents

1	<i>Introduction.....</i>	3
1.1	Evolution of bus devices [E]	3
1.2	Types of KNX bus devices [E].....	5
1.2.1	How each bus device type works	5
2	<i>Data stored in a KNX bus device</i>	7
2.1	KNX bus devices store different types of data in their memory to function correctly.	7
2.2	How does the application program from the KNX manufacturers get into an ETS project?	7
3	<i>System profiles.....</i>	8
3.1	General	8
3.2	Detailed features of KNX System Profiles.....	9
3.2.1	Access control.....	9
3.2.2	KNX Serial number	10
3.2.3	Interface objects.....	10
3.2.4	Memory size.....	10
4	<i>Classical application functions</i>	11
4.1	Dimming with Start/Stop telegram [E]	11
4.2	Dimming with cyclical telegrams [E]	12
4.3	Application function: 'Dimming actuator'.....	13
4.4	Application function: Drive control sensor.....	14
4.5	Application function: Drive control	15
5	<i>Drive control object structure.....</i>	17

Preface

This training document is created for the Basic course. Sections with a heading marked [E] contain important information that is also required for the NextGen course. This includes any sub-sections within them. If a heading does not have [E], it means the content is only for reference and is not mandatory for the NextGen course.

The documentation focuses on the most used configuration mode in KNX, i.e., S-Mode (System Mode). Configuration modes that used to be supported in the past like E-Mode (Easy Mode) are not covered further.

In this chapter, the following abbreviations are used:

PEI = Physical External Interface

BCU = Bus coupling Unit

AM = Application Module

AP = Application Program

TRC = Transceiver

SR = Shift register

DAC = Digital-Analog Converter

1 Introduction

1.1 Evolution of bus devices [E]

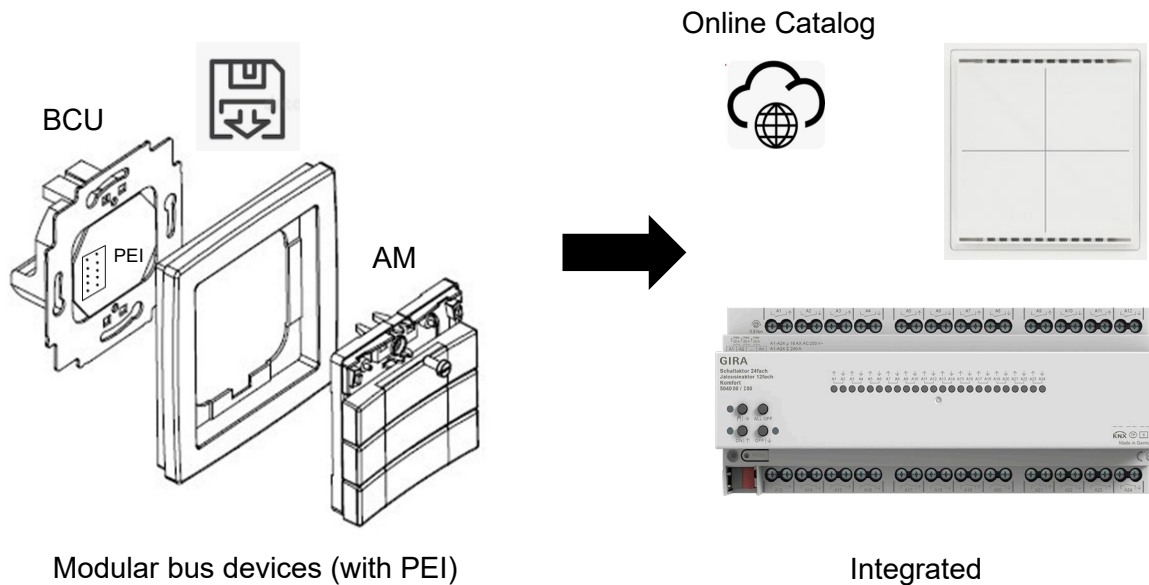


Figure 1: Evolution of bus devices

A bus device is a KNX device that can be configured by means of ETS. When EIB was launched in 1990, it started with just 15 manufacturers and was built around the principle of an **intelligent, decentralized system**. Each EIB bus device originally comprised:

- A Bus Coupling Unit
- An Application Module
- A Physical External Interface with a 10-pin connector to link the BCU and AM

Furthermore, it was agreed to define an Application Program describing the function for each bus device and to commission it using the Engineering Tool Software (ETS).

More than 30 years after its introduction, the EIB standard has evolved into the KNX standard. Today, KNX is supported by over 500 manufacturers and features more than 8,000 certified KNX devices. Alongside this growth, electronic hardware has undergone significant transformation:

- Early devices used through-hole components (manually soldered through circuit boards).
- Today's devices utilize Surface Mounted Devices (SMDs) - compact, for a fully automated placement and soldering process.
- The BCU, once a standalone module, has been miniaturized into two chips, integrated on the application's circuit board.

As a result of this miniaturization, the original modular design (BCU + PEI + AM) has become obsolete. Modern KNX devices typically integrate all components into a single, compact housing, though some manufacturers still use modularity within their own product lines.

A KNX bus device has from the beginning been defined as a combination of:

- A Bus Coupling Unit (BCU)
- An Application Program (AP)
- And usually, an Application Module (AM)

However, in most modern devices, these elements are no longer physically separable. Nowadays bus devices have a compact, integrated design, built into a single housing.

This means that a fully functional KNX bus device - such as a dimming actuator, shutter actuator, or push button - typically consists of the above-mentioned three core components¹.

Each bus device operates independently due to its processor, integrated operating system, and program memory. This **decentralized architecture** is a key characteristic of the KNX system, eliminating the need for a central supervisory unit (e.g., a computer) to manage device communication and operation.

However, if centralized functions such as monitoring and supervision are required, they can be implemented using visualization and control software installed on PCs or tablet computers. This allows for enhanced system management, diagnostics, and remote control, while maintaining the fundamental decentralized nature of KNX.

¹ While most KNX bus devices follow this structure, there are exceptions that do not include an Application Module. Examples are line couplers, IP routers, USB interfaces, logic controllers etc.

1.2 Types of KNX bus devices [E]

KNX bus devices are equipped with a programming button and a programming LED, typically located on the KNX device or the BCU.

- **Programming button:** Activates programming mode for assigning the individual address or is used for diagnostic purposes.
- **Programming LED:** Indicates whether programming mode is ON or OFF.

KNX bus devices are classified into 4 main function classes:

Function Class	Function	Example
System	Bus devices essential for the bus system	Intelligent Power supply, interfaces, couplers
Sensor	Detect and measure physical variables (temperature, voltage, current, etc.).	Thermostats, motion detectors, light sensors
Actuator	Receive bus commands and control outputs (digital or analog).	Light switches, dimmers, shutter drives
Controller	Regulate interaction between sensors and actuators (logical processing).	Logic modules, HVAC controller, logic controller, timer

A single KNX bus device may implement one or more function classes. Examples for combined KNX bus device functionality:

- A push button with a status LED is in principle a sensor (button press) but also an actuator (LED response).
- A KNX actuator gives back status information of the actuator channel to the bus. So, this information goes out to the bus like a “sensor function.”

1.2.1 How each bus device type works

- **System devices** (Power supply², line coupler, interfaces)
 - A KNX line coupler connects the secondary line and the primary line and transmits or blocks the required group addresses as required.
 - A KNX interface e.g., a USB interface is a device that can connect a third-party system to KNX and transfers information.
- **Sensors** (e.g., temperature sensors, motion detectors)
 - The application module of the sensor collects input signals (digital or analog) and transfers them to the BCU. The BCU encodes this data and sends it on the KNX bus.
 - The BCU regularly monitors the input state of the application module.
 - Measure physical variables (temperature, brightness, voltage, etc.).
- **Actuators** (e.g., switch actuator, dimming actuator)
 - Receive telegrams from the KNX bus via the BCU.
 - The BCU decodes the telegram and forwards the command to the application module.

² A standard KNX power supply is not a KNX bus device, because it does not contain a BCU.

- The application module executes the command by controlling an electric load (e.g., switching a relay, dimming a light, adjusting a shutter drive, etc.).
- **Controllers** (e.g., logical modules, timer/event modules)
 - Control interactions between sensors and actuators (e.g., automatically adjusting blinds based on sun position and brightness)
 - Do not have physical inputs or outputs. Typically used for logic operations.

2 Data stored in a KNX bus device

2.1 KNX bus devices store different types of data in their memory to function correctly.

Data Type	Description
System Software (KNX Operating System or "System Stack") (Flash-ROM)	Controls the device's core functions and is identified by a " mask version ". ETS checks this version before configuring the device.
Application program, individual & group addresses, parameters (EEPROM)	Defines the device's function and settings. Stored in rewritable memory.
Temporary values (RAM)	Stores system and application data temporarily (e.g., switching state). Lost if bus power fails, unless saved in non-volatile memory.

2.2 How does the application program from the KNX manufacturers get into an ETS project?

When using the ETS software, the first step is to import the application program from the KNX Online Catalog into the ETS project. The selected application program is configured according to the specifications of the ETS project and then downloaded into the device, defining its functionality.

For example: A KNX push button can only send dimming signals after the correct application program has been downloaded via ETS and the correct parameters have been configured.

The manufacturer code of the application program - which is unique to each KNX manufacturer - and the BCU must match. If they do not, the download will not proceed. ETS checks whether the application program and the manufacturer code match before starting the download into the KNX bus device.

If an application program of a sensor is accidentally downloaded into an actuator from the same manufacturer, two outcomes are possible:

- **Download blocked:** The BCU of the actuator prevents the application program from being downloaded.
- **Download succeeds, but device malfunctions:** The application program is successfully downloaded without an error in ETS, but the actuator will not function.

3 System profiles

3.1 General

KNX has evolved through different system profiles (mask versions) since its introduction. These profiles define the capabilities and limitations of KNX devices.

Feature	System 1 (1991)	System 2/7 (mid 1990s)	System B (Latest)
Mask version	e.g., 0012h	e.g., 0025h, 0705h	e.g., 07B0h
Maximum number of group objects	12	254	65,536
Supports interface objects?	No	Yes	Yes
Supports KNX serial number?	No	Yes	Yes
Supports access control (e.g., BCU Key)?	No	Yes	Yes

Table 1: Overview of KNX System Profiles

The System 1 technology was introduced with the first generation of KNX devices in 1991. Although it is an older system, it remains part of the KNX Standard and can still be implemented in new devices. During the 1990s, additional system profiles were developed, leading to the introduction of System 2 and System 7. To overcome the limitations related to the number of group objects and group addresses, System 7 was further refined and expanded into System B. The table above provides a comparative overview of the most key features of these KNX system profiles.

Compatibility between KNX system profiles:

- System 1 application programs can be downloaded on some System 2 devices (upward compatibility).
- System 2 application programs **CANNOT** be downloaded into System 1 devices.
- This is all managed and handled by ETS automatically.

3.2 Detailed features of KNX system profiles

The features of System 2, System 7, and System B described earlier are further detailed below.

3.2.1 Access control

To access the memory of System 2, System 7, and System B devices (for reading or writing), an authorization key of 4 bytes is required.

Access levels:

- Some levels are reserved for manufacturer access. In such a case, the memory cannot be accessed by system integrators via ETS.
- One level allows access to non-system related memory, which can be locked by entering a "BCU Key" in the ETS Project Details.

BCU Key protection:

- If you enter a BCU Key in the corresponding field, BCUs of System 2 and higher – which support this feature - in this project will be locked with the key.
- If a bus device is locked with a BCU key, it can still send and receive telegrams.
- From this point on, programming is only possible with the correct BCU Key. BCU Keys are stored in the ETS project of the project creator.

Important: When transferring a project to the operating company or another system integrator, the BCU Key must be transferred separately. The BCU Key is not part of the project export. If the BCU Key is lost, configuration or reprogramming of the KNX devices is not possible.

The screenshot shows the 'Details' tab of the ETS software interface for a project named 'MySmart House'. The interface is divided into several sections:

- Left Panel:** Contains input fields for 'Name' (MySmart House), 'Project Type' (Other Residential), 'Project Number', 'Contract Number', and a 'Description' text area.
- Right Panel (Main):**
 - Project Password:** A field with masked characters and a 'Change Password' button.
 - BCU Key:** A text input field, highlighted with a red rectangular box, with a 'Set BCU Key' button next to it.
 - Codepage:** A dropdown menu currently set to 'Windows System Language'.
 - Group Address Style:** Two radio button options: 'Two Level' and 'Three Level' (which is selected).
 - Compatibility:** Two checkboxes: 'Hide extended group address range for plug-ins' (checked) and 'Use low bus communication rate' (unchecked).
- Far Right Panel:** A summary section with a 'Compact now...' button and project statistics: 'Import Date' (10/02/2025 13:56:28), 'Last Modified Date' (22/02/2022 11:55:51), and 'Estimated Project Size' (41.51 MB).

Figure 2: entering a BCU Key in the Project Details in ETS

3.2.2 KNX serial number

System 2, 7, and B devices come with a unique KNX serial number assigned at the factory. This allows reading or writing the individual address **remotely** without pressing the device's programming button.

3.2.3 Interface objects

Interface objects have existed since BCU2. Interface objects contain certain system and application properties (e.g., address table, parameters etc.). ETS can read and modify these properties (e.g., ETS during download) without needing in-depth knowledge of the device's memory structure. System integrators cannot edit interface objects manually, but they can be read using the ETS "Device Editor" App available in the MyKNX Shop. The interface objects are also used when downloading to the devices and improve performance.

A deeper understanding of the system, as explained during the KNX Tutor course, is required to perform this type of manipulation.

3.2.4 Memory size

As one can notice, the memory size of a bus device increases with newer system profiles:

- A System 2 device has a larger memory size than a System 1 device.
- A System 7 device has a larger memory size than a System 2 device.
- System B devices have the highest memory size.

For over a decade, 254 group objects were considered a considerable number. However, with the advancement of touch panels, controllers, and gateways, this limit became insufficient. To address this, System 7 was extended by 1 byte for group addresses and group objects. As a result, the new maximum values are:

- 65,535 for group addresses
- 65,536 for group objects

While these numbers represent theoretical limits, they are rarely reached in practical applications. However, they show the total address capacity of a KNX system, ensuring scalability for future developments.

4 Classical application functions

4.1 Dimming with Start/Stop telegram [E]

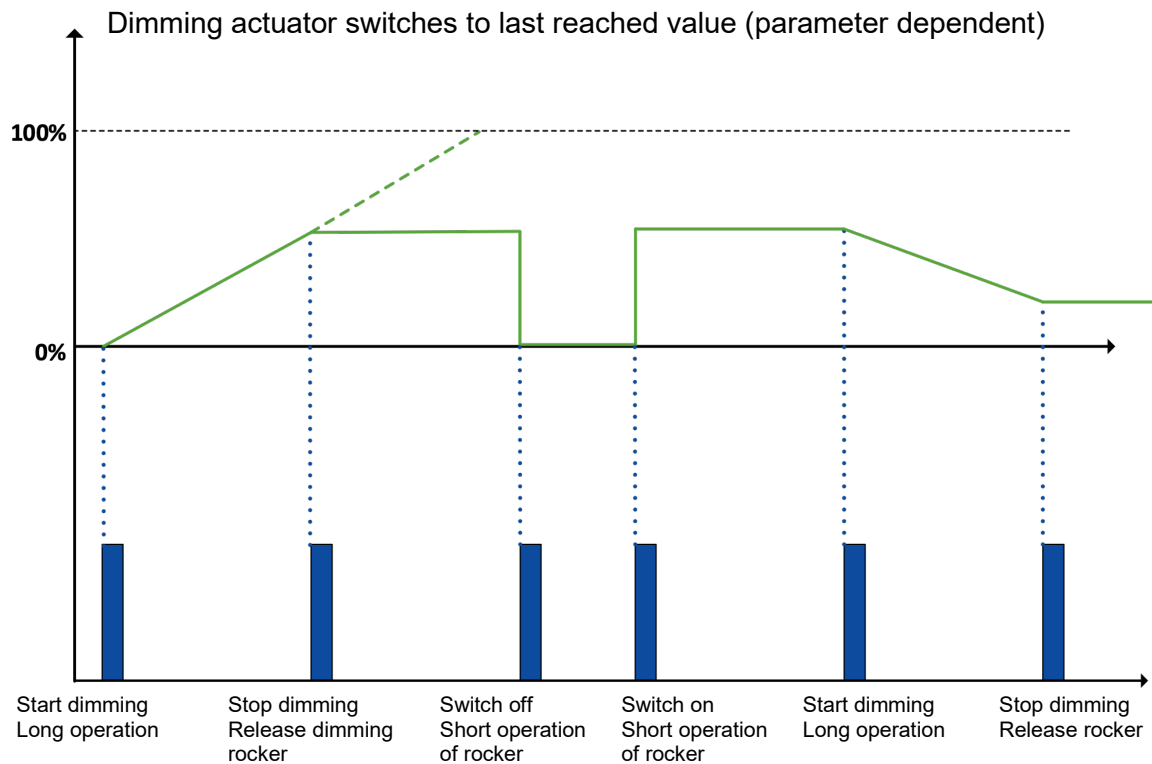


Figure 3: Dimming with start/stop telegram

The operation of the rocker switch determines whether a switching function or a dimming function is activated using the same rocker.

- If the rocker is pressed for a duration shorter than the time parameterized in the application program of the push button (e.g., < 500 ms), a switch telegram is transmitted.
- If the rocker is pressed longer than the configured duration, a "start dimming" telegram is sent.
- Once the rocker is released, a "stop dimming" telegram is transmitted.

To ensure correct execution of functions by the dimming actuator, separate group addresses are used for switching and dimming telegrams.

4.2 Dimming with cyclical telegrams [E]

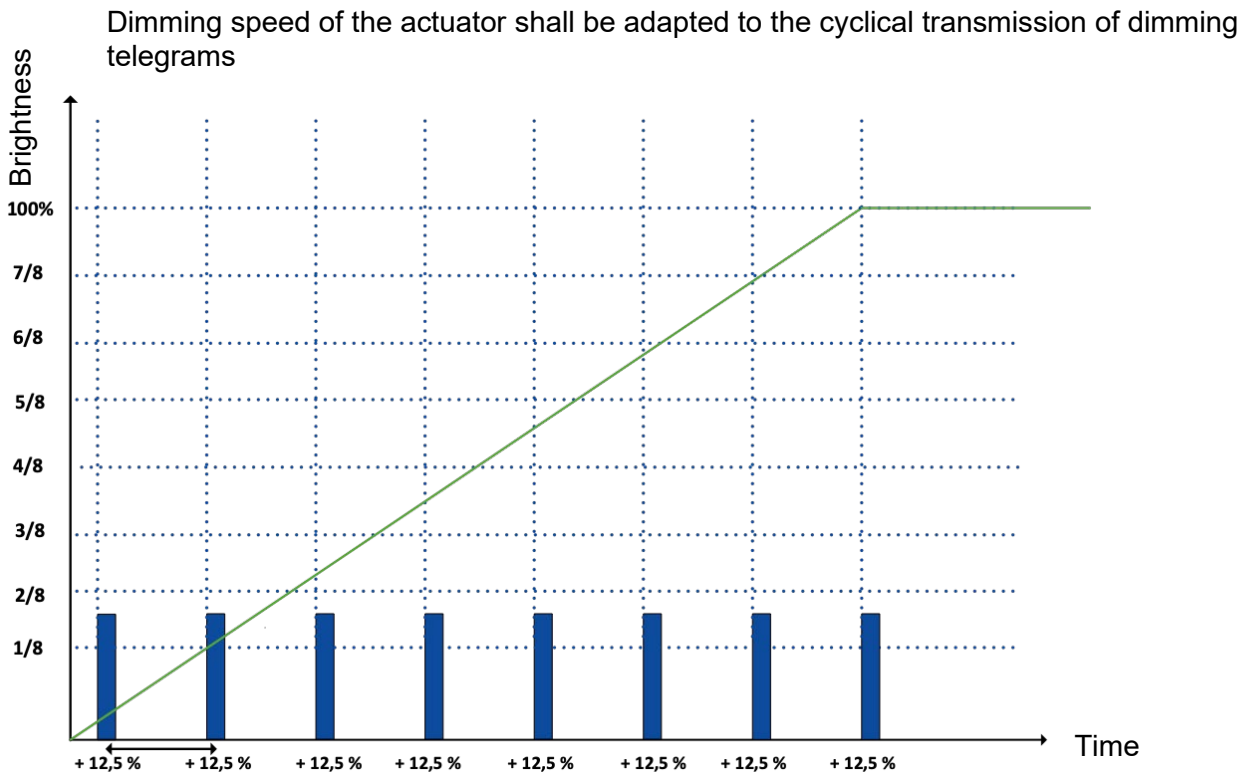


Figure 4: Dimming with cyclical telegrams

In wireless-controlled systems, such as those using infrared (IR) transmitters, transmission signals may be interrupted if an object or person crosses the IR beam. This can lead to missed telegrams, such as the stop telegram, which is crucial for proper dimming control. To mitigate this issue, the **"cyclical dimming"** setting is often enabled during the parameterization of the remote control. In this mode:

- The transmitter repeatedly sends a telegram, such as "increase brightness by 12.5%", ensuring that the dimming actuator continues receiving updates.
- The impact of losing an individual dimming telegram is minimal.
- This approach prevents failures caused by a missing stop telegram, which is otherwise sent only once (as described in section [4.1](#)).

By using cyclical dimming, the reliability of wireless dimming systems is significantly improved, reducing the risk of incomplete dimming commands due to signal interruptions.

4.3 Application function: 'Dimming actuator'

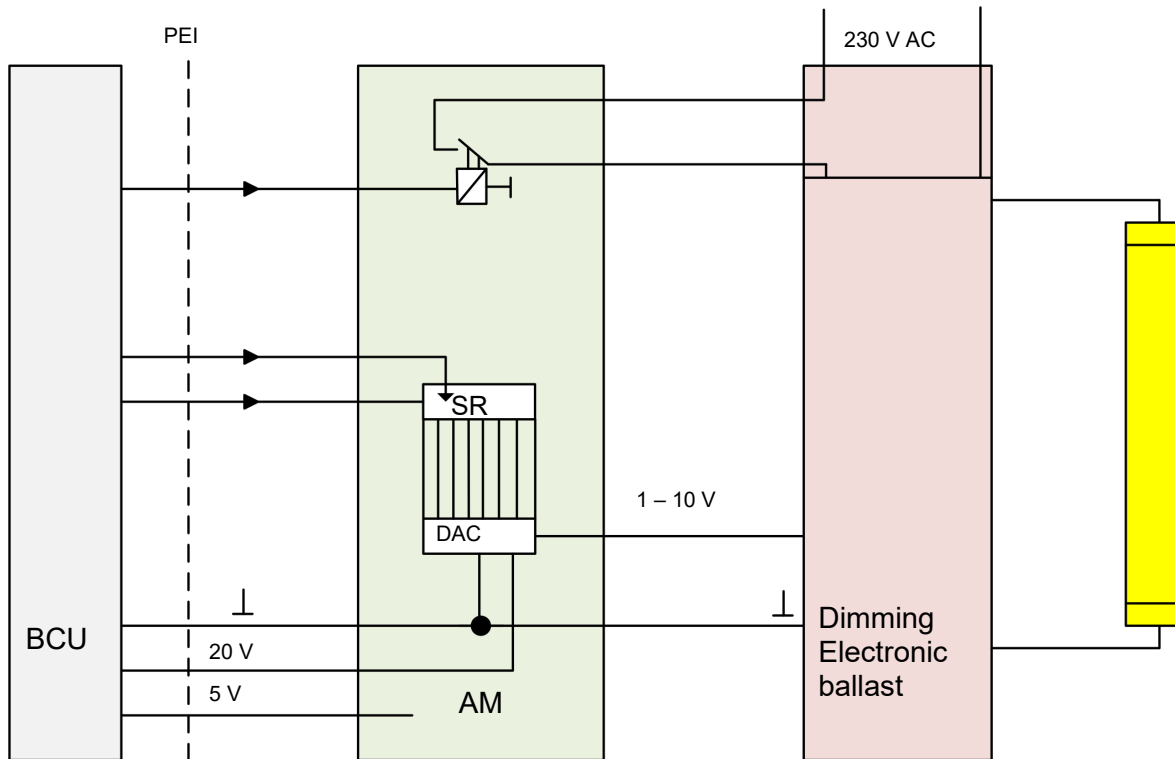


Figure 5: Application function: 'dimming actuator'

The dimming actuator serves as the counterpart to the dimming sensor and is responsible for adjusting light intensity based on the received telegrams. Different types of dimming actuators exist, varying according to the dimming concept, the lamps, and the ballasts used.

In this example, a passive 1–10 V analog interface is shown. However, all dimming actuators share a common feature, i.e., they have a **parameterized dimming speed**, which is exclusively managed by the actuator itself.

Working principle of the dimming actuator:

- The BCU transmits a digital control signal to the application module.
- This signal is then electronically adapted to match the control input of the electronic ballast.
- The electronic ballast utilizes a 1–10 V signal to regulate the brightness of a fluorescent tube.
- A relay in the application module is responsible for switching the mains voltage on or off.

4.4 Application function: Drive control sensor

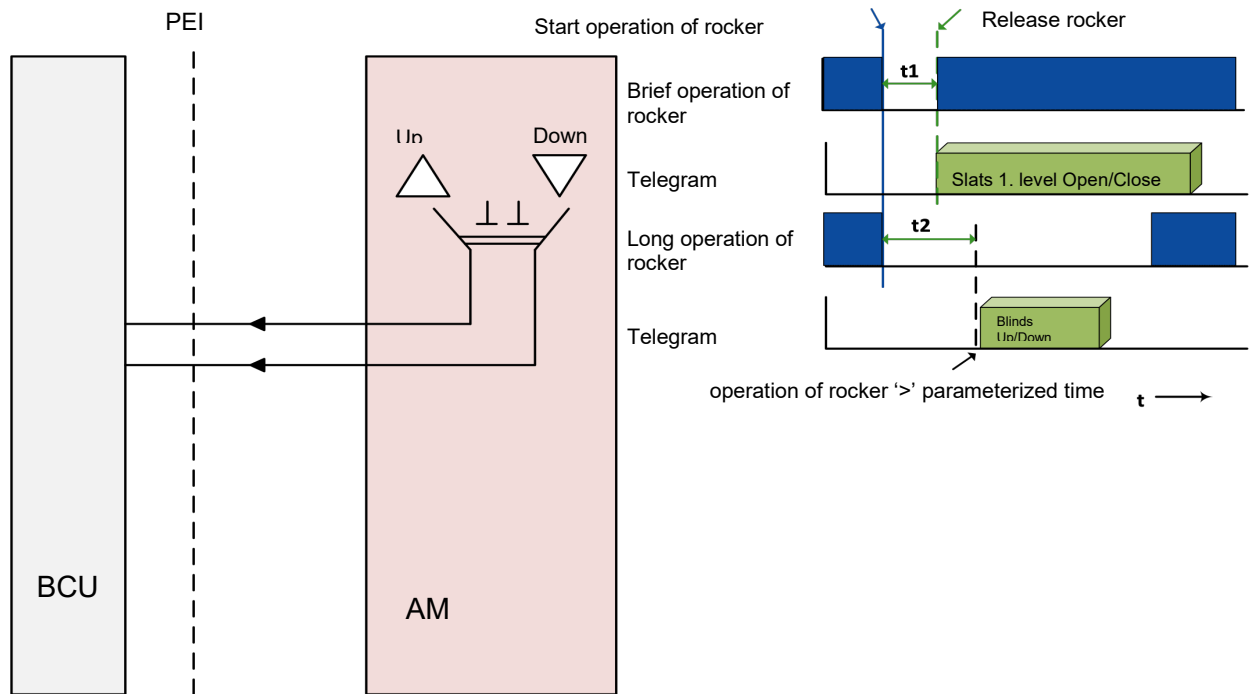


Figure 6: Application function: drive control sensor

The operation of blinds and shutters is similar to dimming control, as it also differentiates between short and long rocker presses.

Operation principles:

- Short press ($< t_2$, e.g., 500 ms):
 - Triggers a "slats step open/close" command (applicable to blinds with adjustable slats).
 - If the drive is already in motion, a short press acts as a stop command.
- Long press ($> t_2$, e.g., 500 ms):
 - Triggers a "blinds up/down" command, which moves the blinds or shutters until they either stop at the limit switch or are stopped again by briefly pressing a button.
- Debouncing time (t_1):
 - Used for push button interfaces and binary inputs to filter out unwanted rapid presses.
 - Typically, not required for standard push buttons.

Key difference from dimming:

Unlike dimming actuators, where the brightness level stops changing when the rocker is released, a **blind/shutter drive continues moving** until:

- A short rocker press stops it.
- The end position is reached.

This behavior makes sense, as blinds and shutters have much **longer travel times** than the time required for dimming from 0% to 100%. For effective blind/shutter control, both short and

long rocker presses must be implemented. Even if slat adjustment is not required, the distinction between short and long operations remains essential for reliable operation.

4.5 Application function: Drive control

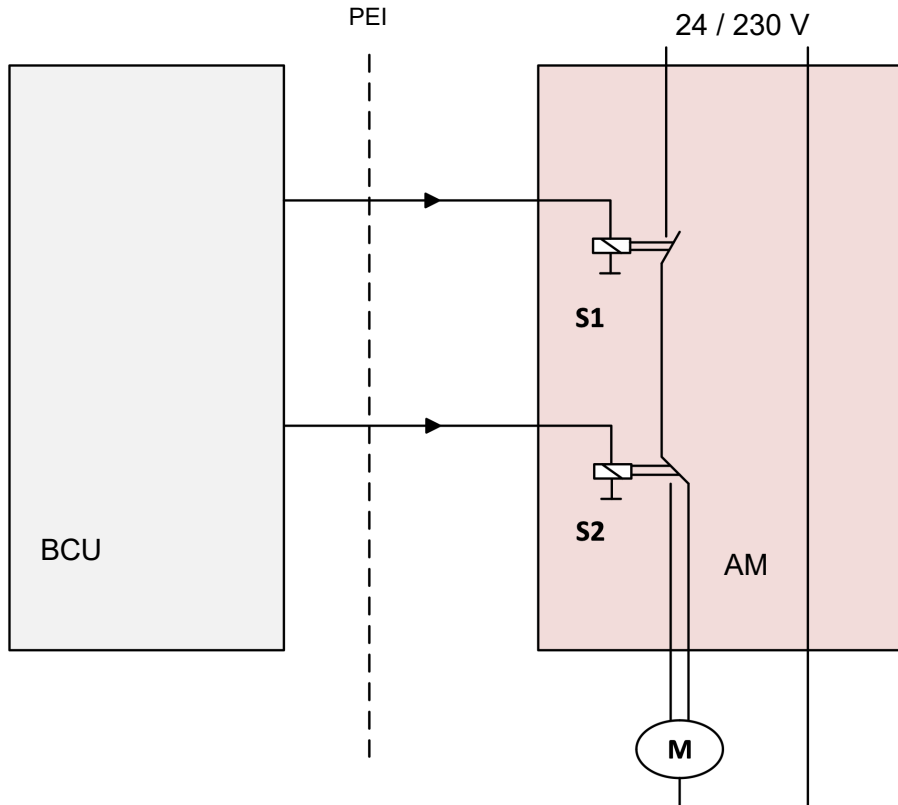


Figure 7: Application function: Drive control

The drive control function manages the movement of shutters and blinds based on received telegrams. The BCU interprets these commands and controls the relays accordingly.

Operation of drive control:

- "Up" or "Down" command:
 - The BCU receives a telegram and activates relay S2, which moves the shutter/blind in the specified direction.
- "Slats Open/Close one step" command:
 - The BCU briefly activates relay S1 for the appropriate duration.
 - If the shutter/blinds are already moving, this command stops it by opening relay S1.
- "Blinds Up/Down" command:
 - The BCU energizes relay S1 for a duration longer than the total movement time required for the shutter/blinds to travel from the top to the bottom (or vice versa).
 - The limit switches of the shutter/blinds ensure the motor stops when the end position is reached, even if the motor remains powered.

A shutter/blinds actuator is not responsible for the safe deactivation of shutter or blinds. Instead, this must always be managed by the motor itself.

5 Drive control object structure

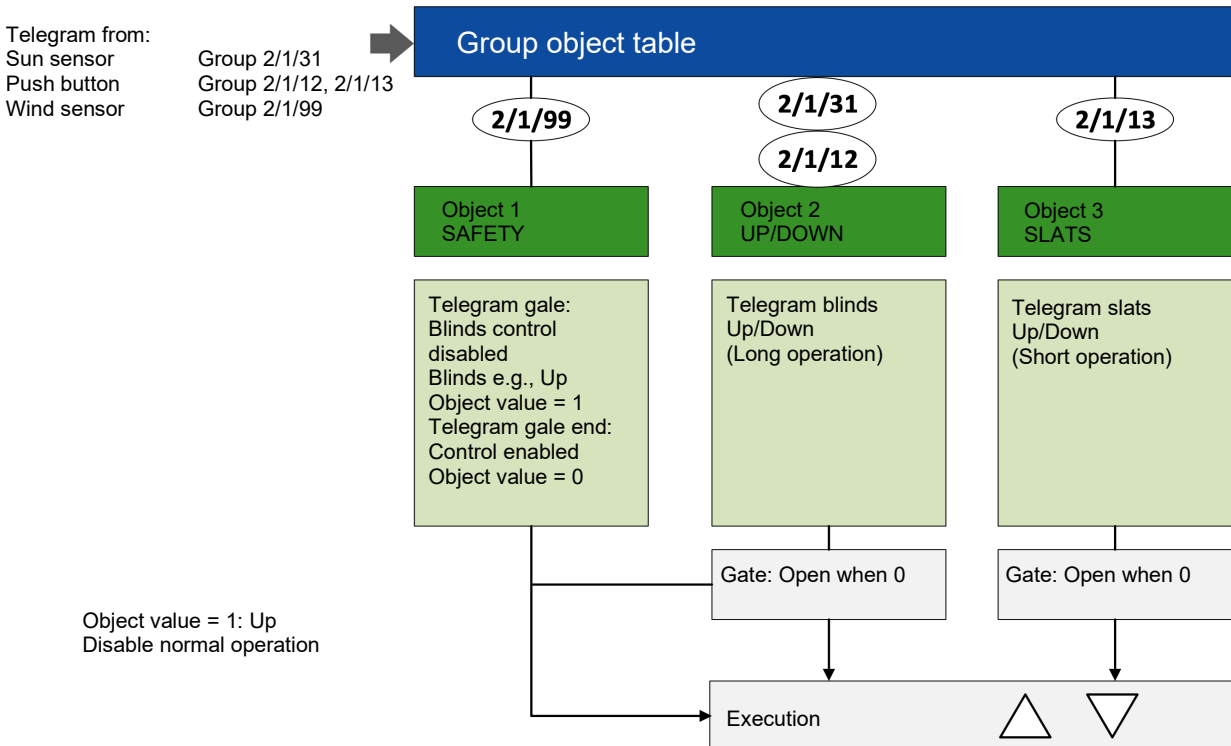


Figure 8: Drive control object structure

The figure above illustrates the basic functionality of a blind/shutter actuator. In addition to standard operations, these actuators can include additional functions, such as safety features.

Basic operation:

- A weather station measuring the brightness of the sun can trigger the "blinds down" command using group address 2/1/31, addressing the "Up/Down" group object and executing the corresponding movement.
- A short press on the push button sends the 2/1/13 telegram to "adjust slats".
- A long press on the push button sends the 2/1/12 telegram, commanding the full opening or closing of the shutter or blinds.

Safety functionality:

- A wind sensor can trigger telegram 2/1/99, which addresses the "Safety" group object.
- In case of strong wind or a developing storm, this telegram moves the blinds up/down (based on parameterization) and disables further manual operation.
- Once the storm subsides, another telegram is sent to restore manual operation.
- However, the actuator does not automatically return the shutter or blinds to their previous position, as it has no information about the duration of the storm or whether lowering the blinds again is necessary.

Modern shutter/blinds actuators offer additional functionalities beyond the basics discussed here. These include integrations with weather stations, among others. These advanced functions are covered in detail in the KNX Advanced course.